

2 - STIMULUS GENERATION

Learning objectives

After completing this lab, you should be able to

- Create a UVM transaction class
- Generate UVM transaction objects in a sequence
- Configure the sequencer to execute the sequence in a desired phase
- Override existing transaction objects in environment with modified transaction class objects

Lab Duration:
30 Minutes

Getting Started

In lab 1, you learned about DSIM and wrote a simple UVM test. But, it didn't do anything. You can experiment with the debugging switches, but nothing more.

In this lab, you will develop a transaction class and a sequence class so that the embedded environment can generate and process transaction objects.

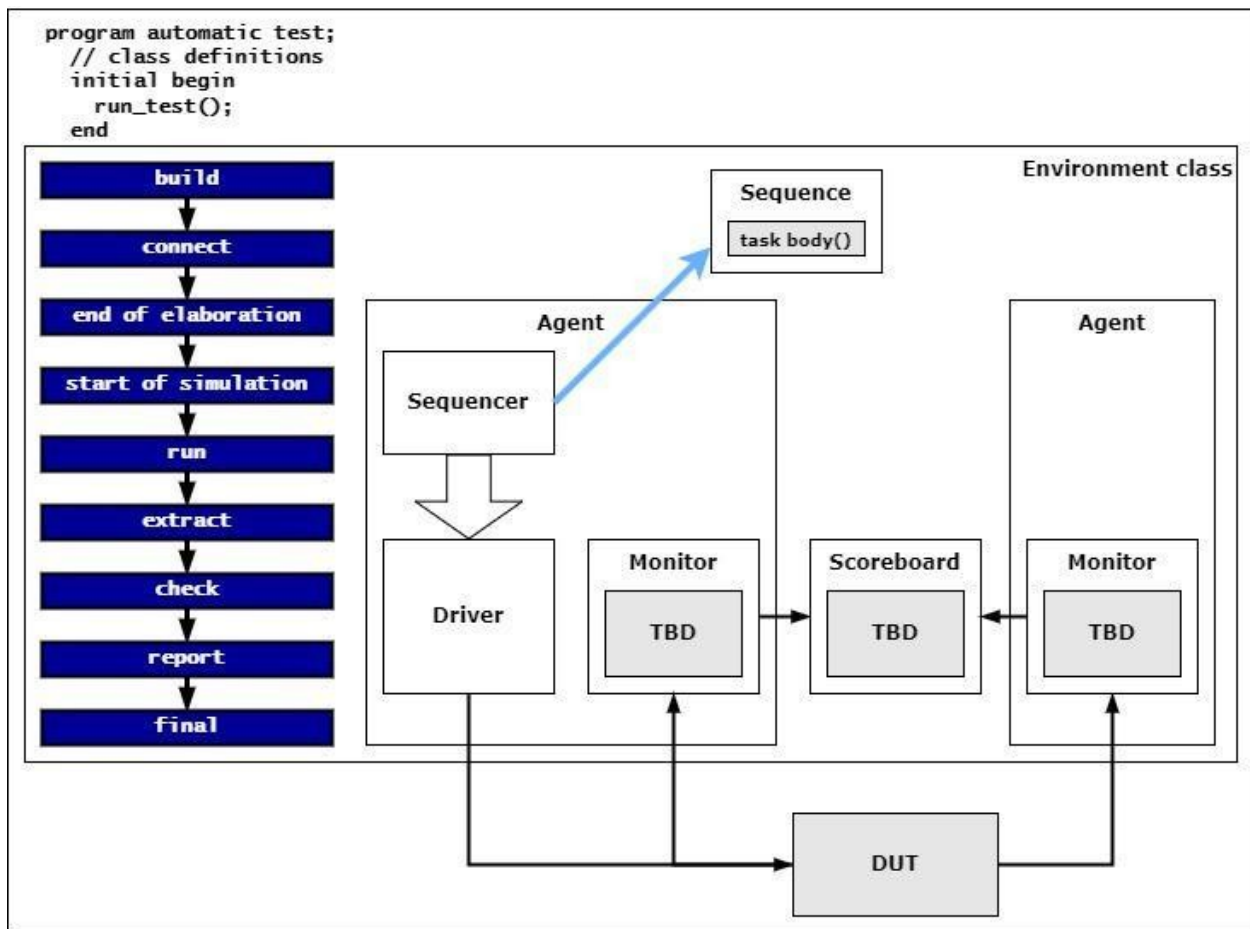


Figure 1. Lab 2 Testbench Architecture

Lab Overview

The work flow for this lab is illustrated as follows.

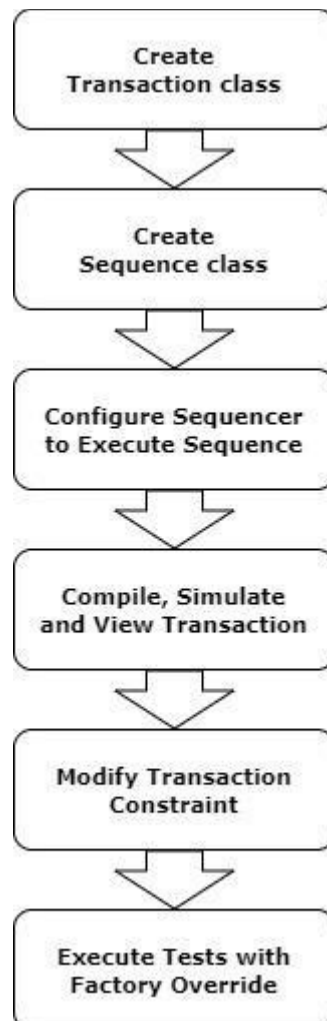


Figure 2. Lab 2 Flow Diagram

Generating UVM Transactions

Task 1. Go into lab2 Working Directory

1. Open a new WSL terminal tab.

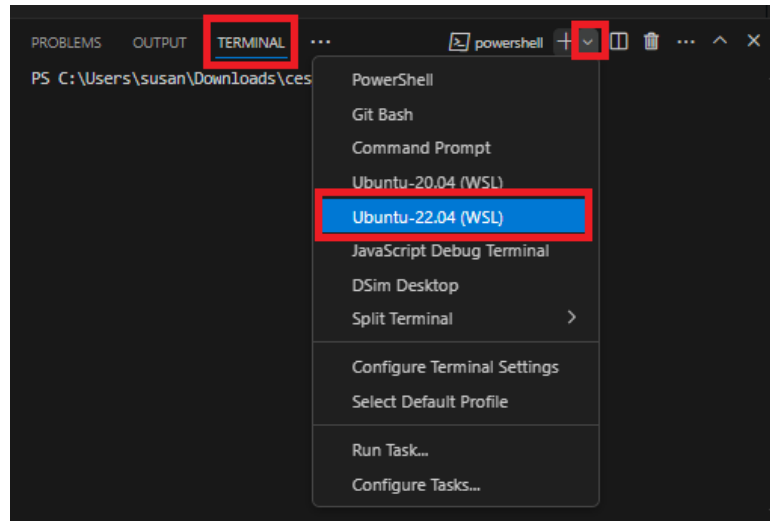


Figure 3. New WSL tab.

Go into **lab2** directory: **ces_uvm-1.2_2021.09/labs/lab2** directory.

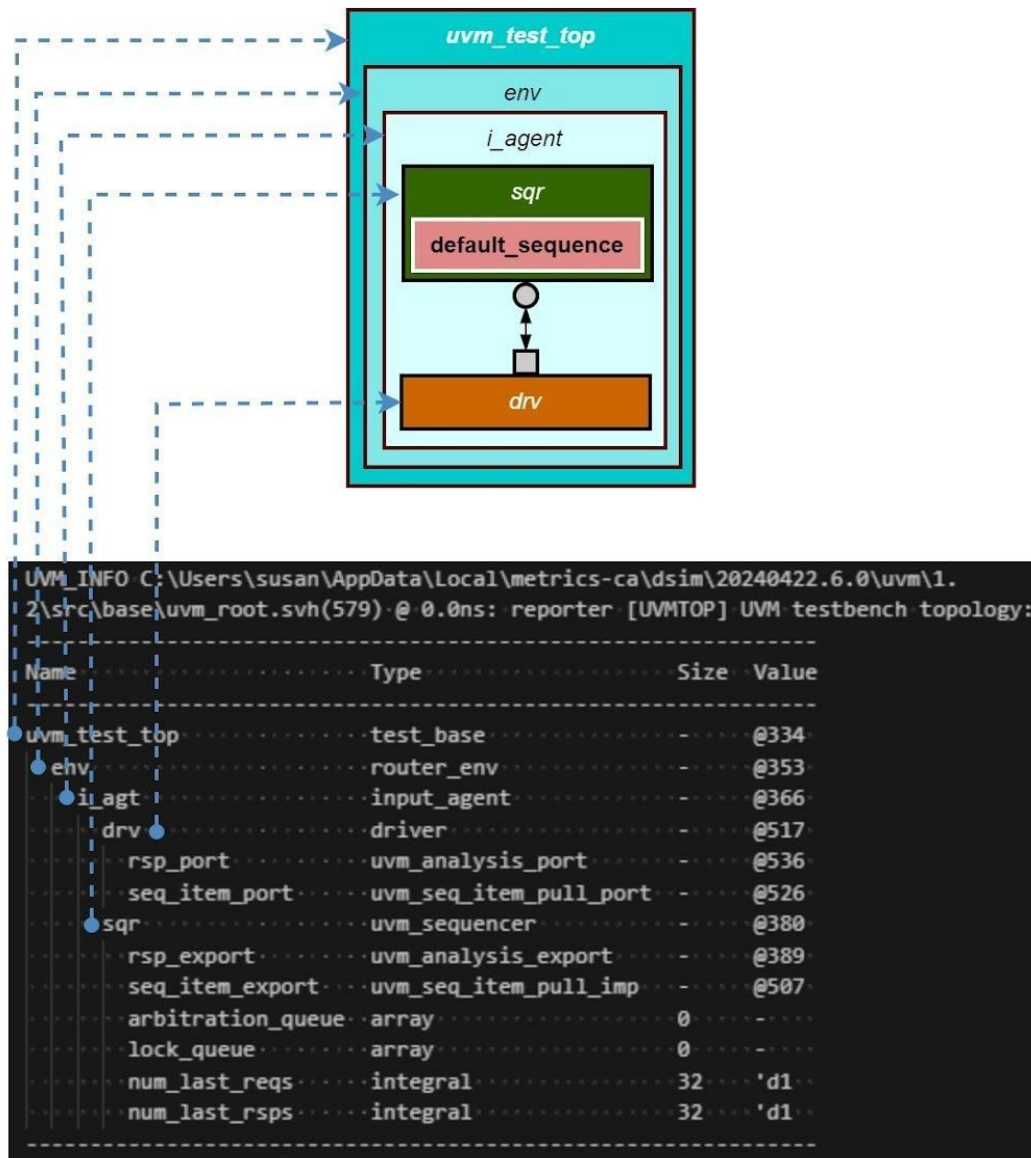
Check the DSImDesktop, if no project has been loaded add the **my_labs_sim.dpf** as your project.

Task 2. Review the Simple UVM Environment

In the previous lab, you constructed an environment in the test class.

This environment consists of a basic UVM component structure: environment (**router_env**), agent (**input_agent**), driver (**driver**) and sequencer (**uvm_sequencer**).

Figure 4. Lab 1 topology.



Task 3. Review Class Organization

Also, in the previous lab, all the class files were included into the program block.

```

program automatic test;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  `include "labs\lab1\packet.sv"
  `include "labs\lab1\driver.sv"
  `include "labs\lab1\input_agent.sv"
  `include "labs\lab1\router_env.sv"
  `include "labs\lab1\test_collection.sv"

  initial begin
    $timeformat(-9, 1, "ns", 10); // Format Verilog time
    run_test();
  end
end

```

This is not an efficient way to manage class files. One downside of this type of implementation is that the **test.sv** file would need to be modified whenever a new test file is created. That's impractical and sometimes impossible.

A better way to manage these files is to organize them as packages. This is done for you. For the labs, you will be making use of three packages located in the **packages** directory: **router_stimulus_pkg**, **router_env_pkg** and **router_test_pkg**.

1. The **router_stimulus_pkg** contains the base transaction class, add the path **<labs/lab2/>**

```

package router_stimulus_pkg;
  import uvm_pkg::*;
  `include "labs\lab2\packet.sv"
endpackage

```

2. The **router_env_pkg** contains the structural component classes, add the path **<labs/lab2/>**

```
package router_env_pkg;
  import uvm_pkg::*;
  import router_stimulus_pkg::*;
  `include "labs\lab2\driver.sv"
  `include "labs\lab2\input_agent.sv"
  `include "labs\lab2\router_env.sv"
endpackage
```

3. The **router_test_pkg** contains the test specific classes, add the path **<labs/lab2/>**

```
package router_test_pkg;
  import uvm_pkg::*;
  import router_stimulus_pkg::*;
  import router_env_pkg::*;
  `include "labs\lab2\test_collection.sv"
endpackage
```

Once the files are organized this way, you will find your management of files becomes easier.

Task DSim. Adding DSim Files and Simulation Configs

1. Into the **lab2** folder, rename the **test.sv** file as **test_2.sv** file.
2. Open the **test_2.sv** file with an editor.
3. Rename the **program automatic test** as **program automatic test_2**.

Include `<`include "uvm_macros.svh">` below `<import uvm_pkg::*;>` and save the file.

4. Open the **verif_filelist** file with an editor.
5. Add the following code lines and save the file.

```
rtl\router_test_top.sv
labs\lab1\test.sv
labs\lab2\packages\router_stimulus_pkg.sv
labs\lab2\packages\router_env_pkg.sv
labs\lab2\packages\router_test_pkg.sv
labs\lab2\test_2.sv
```

6. Hover over the **SIMULATION CONFIGURATION** drop-down menu and click the + button.

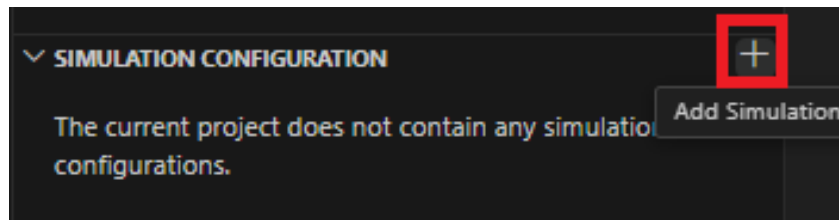


Figure 19. Add a simulation configuration.

Fill and save the configuration tab with the name **lab2_sim**, and the next options:

```
-top work.test_2 -uvm 1.2 -L work +acc+b -waves
lab2_test.mxd -dump-aggr -sv_seed random -timescale
1ns/1ps +UVM_TIMEOUT=10000 +UVM_VERBOSITY=UVM_MEDIUM
+UVM_NO_DEPRECATED +UVM_OBJECT_MUST_HAVE_CONSTRUCTOR
+UVM_TESTNAME=test_base
```

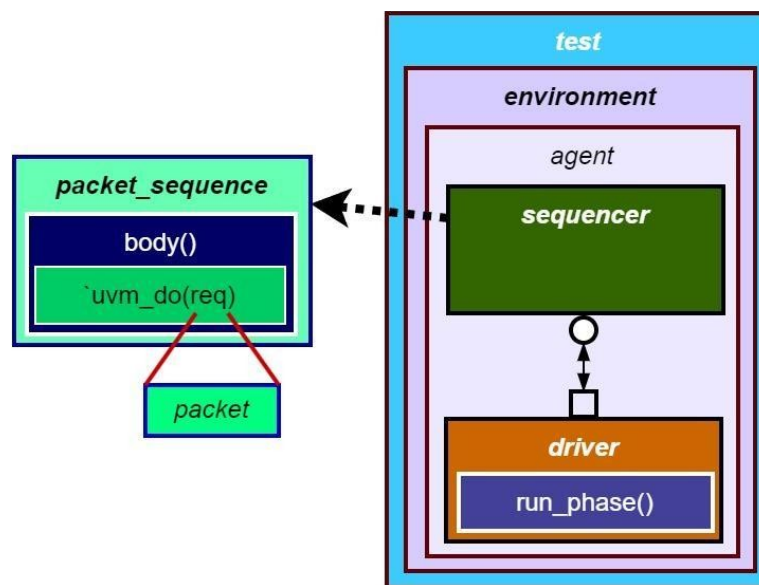
Task 4. Update Program Block

With the packages provided, you will no longer need to include the class files into the program block.

1. Open the **test.sv** file with an editor.
2. Replace the include statements with import of the **router_test_pkg** and save the file.
3. Recompile and then run the **lab2_sim** through the DSIM desktop.

You should see the same result as the previous lab. The reorganization of the class files does not change the behavior of the test.

In the execution of the test, no stimulus was generated. You need a transaction class (**packet**) and a sequence class (**packet_sequence**) that constructs and randomizes objects. Then, configure the sequencer to execute the sequence in a desired phase so that transaction objects created in the sequence can be passed on to the driver in the desired task phase.



Task 5. Register the Packet Class into Factory

1. Open the **packet.sv** file with an editor.
2. Look for : “// **Lab 2: Task 5, Step 2**” and do the following:
 - Register the class into the UVM factory.
 - Enable all fields to be processed (except the sa compare field, see comment)
3. Look for : “// **Lab 2: Task 5, Step 3**”
 - Define the constructor method

Note: *There is a major difference between transaction class constructor and component class constructor. In the transaction class constructor, there is no parent component handle in the argument.*

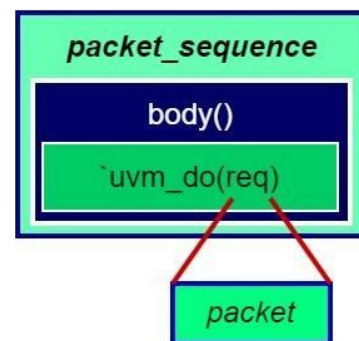
4. Save and close the file.

Task 6. Generate Stimulus Transaction

1. Open the **packet_sequence.sv** file with an editor.
2. Look for “// **Lab 2: Task 6, Step 2**”
 - Enter the **class declaration**

You are required to parameterize the sequence class with the sequence_item class that the sequence is designed to generate for the driver to process

When implementing the sequence class, one must define the **body()** task in which stimulus are generated.



3. Look for “// **Lab 2: Task 6, Step 3**”

Create the **body()** method as described in the comment.

One BIG caution before leaving the sequence class - UVM-1.2 was released on June 24, 2014. Unfortunately, some of the mechanisms implemented in UVM-1.1 have been deprecated and will not compile with UVM-1.2. One of these is the objection mechanism in sequence classes. Please read the comments as directed by the following two steps.

4. Locate the constructor **new()** method and read the comments in the method.
5. Locate the **pre_start()** and **post_start()** methods and read the comments.

Please make sure you understand what's going with the objection handling. This will save you a lot of headaches later.

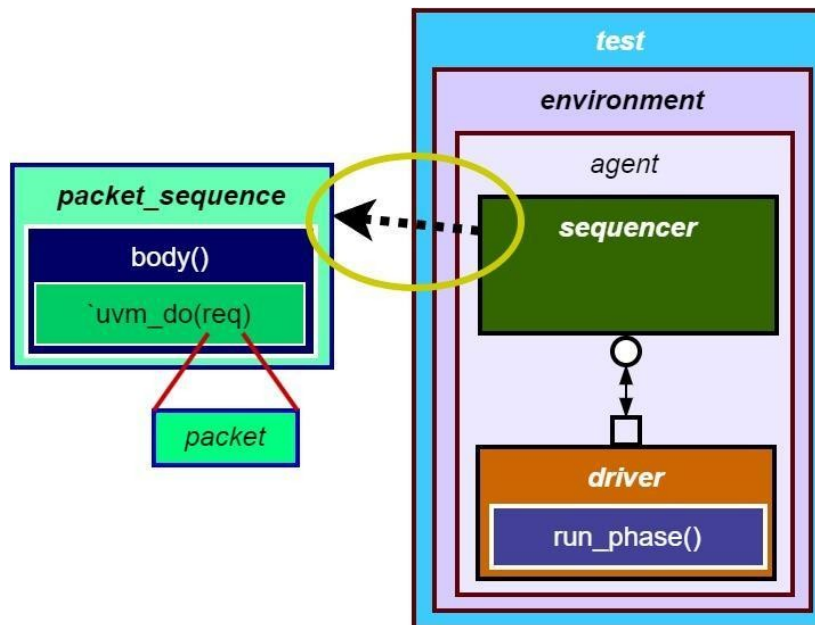
6. Save and close the file.

Task 7. Configure Sequencer to execute Sequence

With the **packet** and the **packet_sequence** classes completed, you now have the mechanism for generating stimulus in UVM. The last step in this process is to configure the sequencer to execute the sequence.

1. Open the **router_env.sv** file with an editor
2. Look for "**// Lab 2: Task 7, Step 2**"

Set the **i_gnt**'s sequencer (**sqr**) to execute **packet_sequence** as the default sequence in the **main phase**.



3. Save and close the file

Task 8. Include Sequence Class in Stimulus Package

The sequence class needs to be compiled. The proper place for this is as part of the stimulus package.

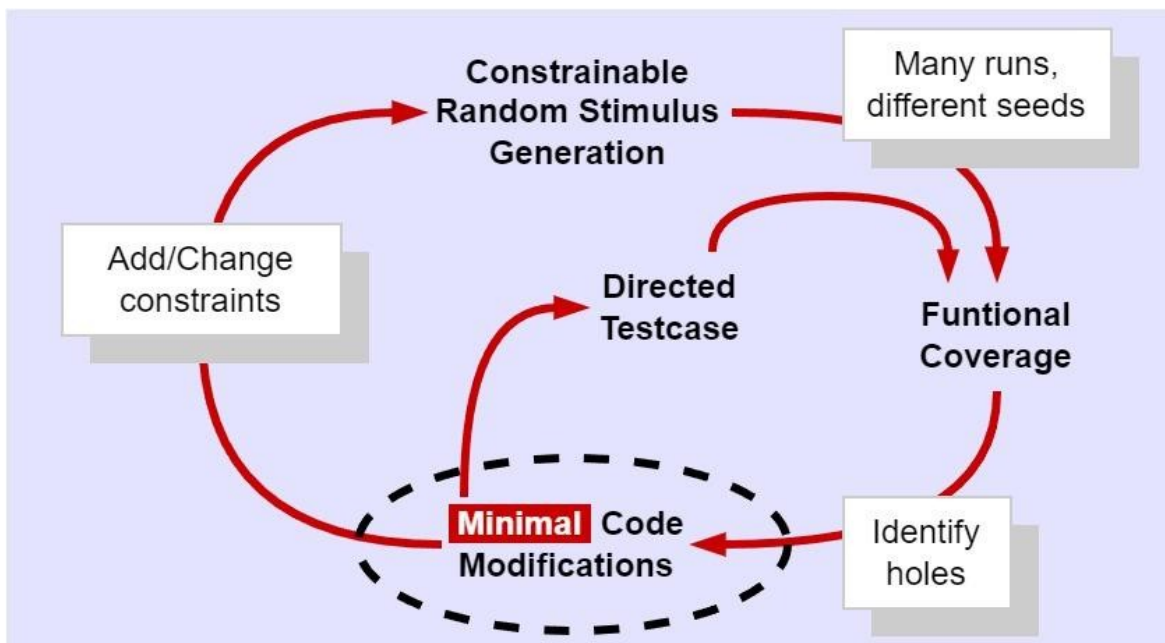
1. Open **packages/router_stimulus_pkg.sv** file with an editor.
2. Add an include statement for the **packet_sequence.sv** file.

*Not forget to use the path **<labs/lab2/>**
3. Save and close the file.
4. Recompile and then run the **lab2_sim** through the DSIM desktop.
5. Take a look at the simulation result in the **dsim.log** file.

You should see that ten packets were printed. You are now able to generate stimulus for the driver to process.

Task 9. Execute Tests with Override

The goal of UVM is to simplify the development and execution of individual tests. One way to achieve this goal in UVM is to make use of the factory override feature.



In this task, you will constraint the destination address (**da**) for the test to 3.

1. Open **packet_da_3.sv** file with an editor
2. Look for the **ToDo** comments and write a constraint to set destination address (**da**) to 3

3. Save and close the file

Task 10. Include Class in Stimulus Package

1. Open **packages/router_stimulus_pkg.sv** file with an editor
2. Add an include statement for the **packet_da_3.sv** file.

*Not forget to use the path **<labs/lab2/>**

3. Save and close the file

Task 11. Create a Test to Use the Modified Packet Class

1. Open **test_collection.sv** file with an editor
2. Look for "**// Lab 2: Task 11, Step 2**"
3. Use instance override to configure the sequence to use **packet_da_3** instead of **packet**.
4. Save and close the file

Task 12. Compile And Stimulate with New Test

1. Recompile and then run the **lab2_sim** through the DSIM desktop. Rename the log as **dsim_fully_rand.log**.
2. Hover over the **lab2_sim** from the **SIMULATION CONFIGURATION** click right click it, choose "*Configure Simulation*".

Replace **+UVM_TESTNAME=test_base** with **+UVM_TESTNAME=test_da_3_inst** and save the changes.

Run the **lab2_sim** through the DSIM desktop.

3. Open a WSL terminal tab.

Go into **ces_uvm-1.2_2021.09** directory, use the next command:

```
> grep -w ad dsim.log
```

Check to see that the destination addresses for these packets are all 3.

4. Open the **dsim.log** file and check to see that the Factory Configuration report displays the **Instance Overrides** correctly.
5. Contrast the actual **dsim.log** with the **dsim_fully_rand.log**

Task 13. Compile And Stimulate with New Test

For factory override, you actually don't need to write a test. You can do the same thing with a run-time switch.

1. Hover over the lab2_sim from the **SIMULATION CONFIGURATION** click right click it, choose "*Configure Simulation*".

Try instance override (you should type out the full path instead of wildcard) by adding the next switch and save the changes:

```
+uvm_set_inst_override=packet,packet_da_3,\*.req
```

2. Hover over the lab2_sim from the **SIMULATION CONFIGURATION** click right click it, choose "*Configure Simulation*".

Remove the switch of the last step and try with the next one:

```
+uvm_set_type_override=packet,packet_da_3
```

So as long as you have a path to an object creation, you can override any of the classes that you registered into the factory. For the user, it means that once you have created all the classes that you need, you will not need to write another test or re-compile. You can use the run-time switches to pick and choose the overrides to accomplish what you want to test.

You are done with lab 2!
Solutions

test_2.sv

```
program automatic test_2;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import router_test_pkg::*;

  initial begin
    $timeformat(-9, 1, "ns", 10);
    run_test();
  end

endprogram
```

test_collection.sv

```

class test_base extends uvm_test;
  `uvm_component_utils(test_base)

  router_env env;

  function new(string name, uvm_component parent);
    super.new(name, parent);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
  endfunction: new

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    env = router_env::type_id::create("env", this);
  endfunction: build_phase

  virtual function void final_phase(uvm_phase phase);
    super.final_phase(phase);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    if (uvm_report_enabled(UVM_MEDIUM, UVM_INFO, "TOPOLOGY")) begin
      uvm_root::get().print_topology();
    end
    if (uvm_report_enabled(UVM_MEDIUM, UVM_INFO, "FACTORY")) begin
      uvm_factory::get().print();
    end
  endfunction: final_phase
endclass: test_base

class test_da_3_inst extends test_base;
  `uvm_component_utils(test_da_3_inst)

  function new(string name, uvm_component parent);
    super.new(name, parent);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
  endfunction: new

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);

    set inst override bv tvne("env.i aat.sar.packet sequence.read".

```

packet.sv

```

class packet extends uvm_sequence_item;

    rand bit [3:0] sa, da;
    rand bit[7:0] payload[$];

    `uvm_object_utils_begin(packet)
        `uvm_field_int(sa, UVM_ALL_ON | UVM_NOCOMPARE)
        `uvm_field_int(da, UVM_ALL_ON)
        `uvm_field_queue_int(payload, UVM_ALL_ON)
    `uvm_object_utils_end

    constraint valid {
        payload.size inside {[1:10]};
    }

    function new(string name = "packet");
        super.new(name);
        `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    endfunction: new

```

```

class router_env extends uvm_env;

    input_agent i_agt;

    `uvm_component_utils(router_env)

    function new(string name, uvm_component parent);
        super.new(name, parent);
        `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    endfunction: new

    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);

        i_agt = input_agent::type_id::create("i_agt", this);

        uvm_config_db #(uvm_object_wrapper)::set(this,
            "i_agt.sqr.main_phase", "default_sequence",
            packet_sequence::get_type());

```

```

class packet_sequence extends uvm_sequence #(packet);

// Register class into UVM factory.
`uvm_object_utils(packet_sequence)

function new(string name = "packet_sequence");
    super.new(name);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);

    `ifndef UVM_VERSION_1_1
        set_automatic_phase_objection(1);
    `endif
endfunction: new

virtual task body();
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    repeat (10) begin
        `uvm_do(req);
    end
endtask

`ifdef UVM_VERSION_1_1

virtual task pre_start();
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    if ((get_parent_sequence() == null) && (starting_phase != null)) begin
        starting_phase.raise_objection(this);
    end
endtask: pre_start

virtual task post_start();
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    if ((get_parent_sequence() == null) && (starting_phase != null)) begin
        starting_phase.drop_objection(this);
    end
endtask: post_start

```